TECHNICAL WORKSHOP · 45 MIN + Q&A

Wealth Agent

A deep research agent built with LangChain, LangGraph, Deep Agents, and LangSmith

---

It reads brokerage accounts, spending data, and then says what to move using an investment policy, and shows where every number came from.

ADITHYA NATARAJAN

Senior Software Engineer

DEEP AGENTS · LANGGRAPH · LANGSMITH

Account data in. A defensible client report out.

A wealth memo is a report that a wealth advisor writes before a client review.

WHAT GOES IN

01 Positions and balances - Trading MCP server

02 Six months of transactions - Banking MCP server

03 An investment policy - targets, bands, cash floor

04 Market context - live web search, pages stored

WHAT COMES OUT · ONE LINE OF THE MEMO

TRIM AAPL -$11,759.48

Information Technology is 34.71% of equity against a 25% target with a 5-point band.

Click any number for the tool behind it.

Hours of mechanical assembly per review become about four minutes and a dollar.

Before every client review, a wealth advisor spends hours researching and putting the same numbers together manually.

The hours come back if nobody can check it

A wrong figure in a client-facing memo is not a rough edge. It is a compliance finding.

SO BEFORE ANYONE ACTS ON IT

01 Where did every number come from?

02 Which source supports each market claim?

03 What stops an unsafe change to the account?

WHAT ONE MEMO ASKS A REVIEWER TO TRUST

142

figures and claims in one memo

An agent that saves an analyst three hours and costs a reviewer three hours has saved nobody anything.

`Fluency is visible. Proof is engineered.`

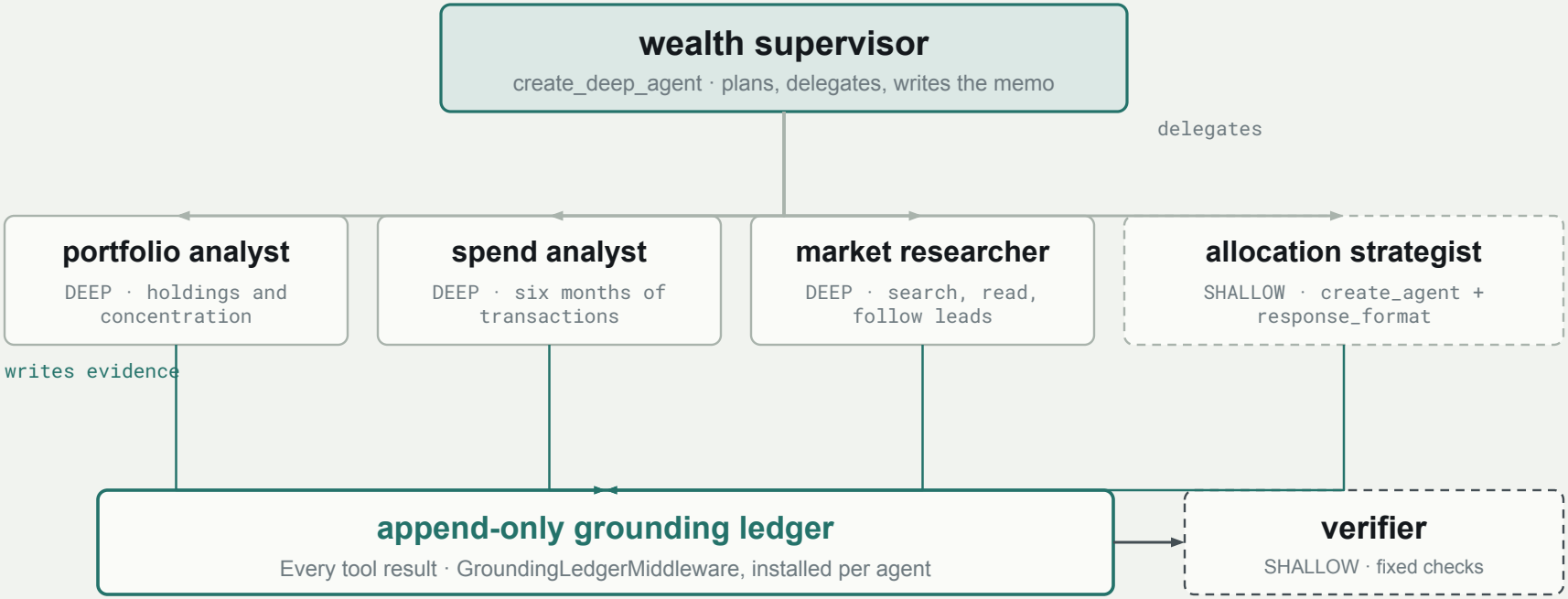
Answering all three mechanically, on every run, without a human re-deriving anything — that is what the rest of this session builds.

Use each LangChain layer for the job it does best

LAYER	WHAT IT ADDS	WHAT THE WEALTH AGENT USES FROM IT
LangChain	Agent framework	create_agent, @tool, middleware, typed structured output
LangGraph	Durable runtime	InMemorySaver checkpointing, interrupts, resume
Deep Agents	Opinionated harness	create_deep_agent — planning, subagents, filesystem, skills
LangSmith	Engineering lifecycle	Traces, datasets, experiments, custom evaluators

Start at the smallest layer that solves the problem.

A subagent is a context window with a job



Deep where the work is open-ended. Shallow where there is exactly one right answer.

Agentic decisions. Deterministic checks.

THE MODEL DECIDES

Plan and delegate

Which specialist should investigate next

Research

Which sources and follow-up leads matter

Explain trade-offs

How to communicate competing policy constraints

CODE ENFORCES

Compute

Portfolio weights, drift, runway, trade amounts

Record

Every tool result enters an append-only ledger, via middleware

Gate

Citations, figures, permissions, and release thresholds

`create_deep_agent for open-ended work` · `create_agent + response_format for bounded work`

Two memos. Which one would you ship?

01

Open the naive memo

Same data, same question, no verification. Let the room read its strongest recommendation.

02

Ask what would earn approval

It is the best-written of the three and it attributes none of its twenty market claims.

03

Open the grounded memo

Click one number: the trace shows the tool that produced it and the agent holding it.

SAME DATA · SAME QUESTION

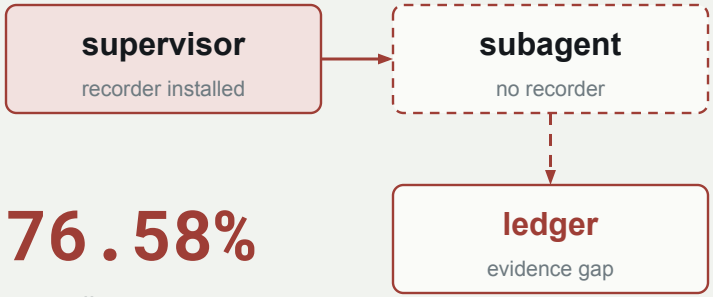
NAIVE	GROUNDED
0	CLICK
citations checked	a figure to open its evidence

The persuasive memo is the least defensible one.

The middleware never reached the subagents

$18,420.55 came from *get_account_balances* and the verifier called it unsupported.

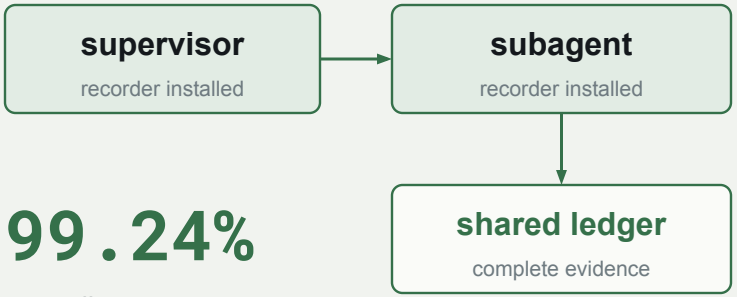
BEFORE



76.58%

grounding

AFTER



99.24%

grounding

Nothing errored. A cross-cutting control installed in one place failed silently everywhere else.

Use a judge only where code cannot answer

01

Deterministic

Citations + numeric grounding

18 / 20

labeled defects caught · $0

02

LLM judge

Semantic support

2 gaps

wrong denominator · rounding

03

Trajectory

What the agent actually did

required

tools · approvals · budgets

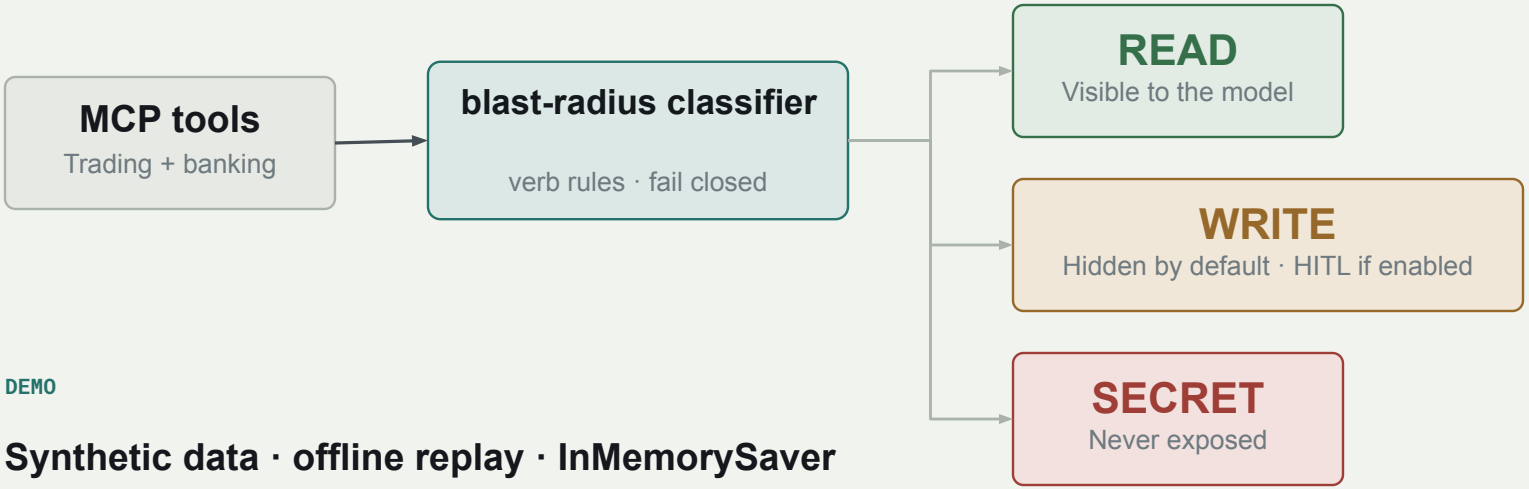
A right answer reached by an unsafe path is still a failed agent.

An evaluator is an application. Measure it first.

JUDGE PROMPT	AGREEMENT	BLESSED BAD	FLAGGED GOOD
v1 · reasonable first draft	85%	2	1
v2 · strict after reading misses	85%	0	3
v3 · calibrated	95%	0	1

v1 and v2 have the same headline score. Only the error direction shows that v2 eliminated the dangerous false accepts.

Two switches: what it sees, what it may do



DEMO

Synthetic data · offline replay · InMemorySaver

PRODUCTION

Durable checkpoint store · idempotency keys · tenant isolation · append-only audit

1 Record evidence at the moment it arrives.

Middleware makes the audit trail the default, not an optional habit in every tool.

---

2 Measure the judge before you trust it.

A score is only useful when you understand the evaluator’s own error distribution.

Both are why the customer's reviewer checks 2 flagged claims instead of re-deriving 142.